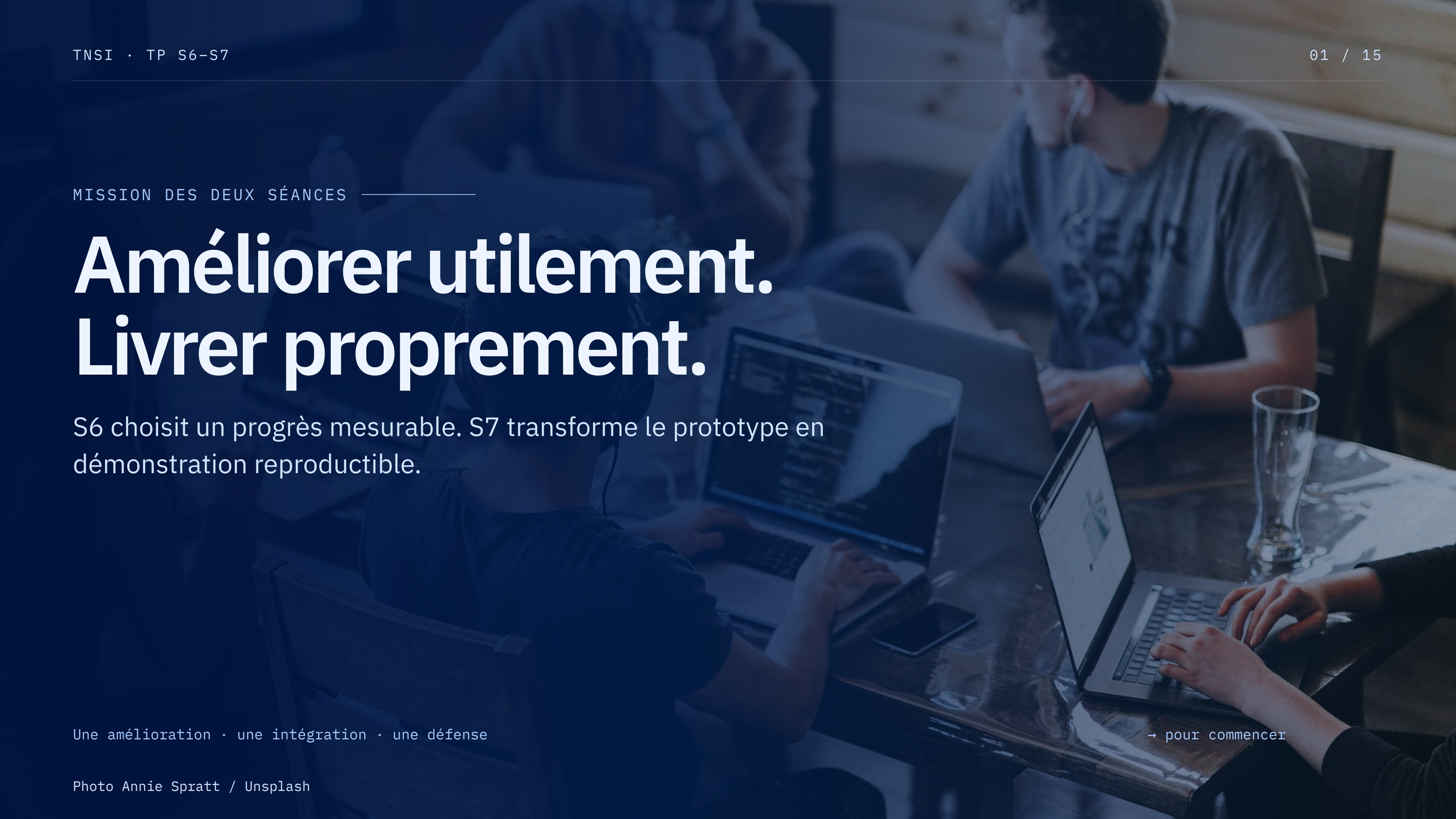


MISSION DES DEUX SÉANCES

Améliorer utilement. Livrer proprement.

S6 choisit un progrès mesurable. S7 transforme le prototype en démonstration reproductible.

Une amélioration · une intégration · une défense

→ pour commencer

Photo Annie Spratt / Unsplash

LE PLAN

Deux séances, quatre jalons visibles.

01 · S6

Choisir

Un problème concret et une mesure avant/après.

02 · S6

Améliorer

Une évolution petite, testée et documentée.

03 · S7

Intégrer

Installation fraîche et parcours de bout en bout.

04 · S7

Répéter

Démo, limites et défense individuelle.

SE REPÉRER AVANT DE CODER

Le prochain progrès doit être visible par un utilisateur.



Une personne code, une autre vérifie, une troisième explique.

S6 · OBSERVER

Choisir un défaut reproductible.

Reprendre les questions et traces des séances précédentes. Conserver un résultat avant toute modification.

S6 · AMÉLIORER

Changer une seule responsabilité.

Écrire le test qui manque, modifier la capacité concernée, puis rejouer tous les scénarios.

S7 · LIVRER

Faire repartir le projet chez un autre membre.

Nouveau dossier, configuration décrite, projections reconstruites et démonstration chronométrée.

CHOISIR L’AMÉLIORATION

Traitez une faiblesse observée, pas une technologie à placer.

QUALITÉ

Réponses

Chunking, citations, seuil, question hybride ou abstention.

CAPACITÉ

Parcours

Nouveau tool, relation graphe, filtre ou écran utile.

ROBUSTESSE

Confiance

Tests, validation, erreurs ou démarrage reproductible.

- ✓ Une phrase de problème.
- ✓ Une mesure avant.
- ✓ Une modification limitée.
- ✓ Une limite assumée.

MESURER AVANT DE CODER

Gardez une preuve du point de départ.

1

Choisir un scénario

Une question, une action ou un démarrage qui pose problème.

2

Capturer le résultat

Sortie, durée, sources, erreur ou nombre d’étapes.

3

Fixer le succès

Un critère observable qui permettra la comparaison.

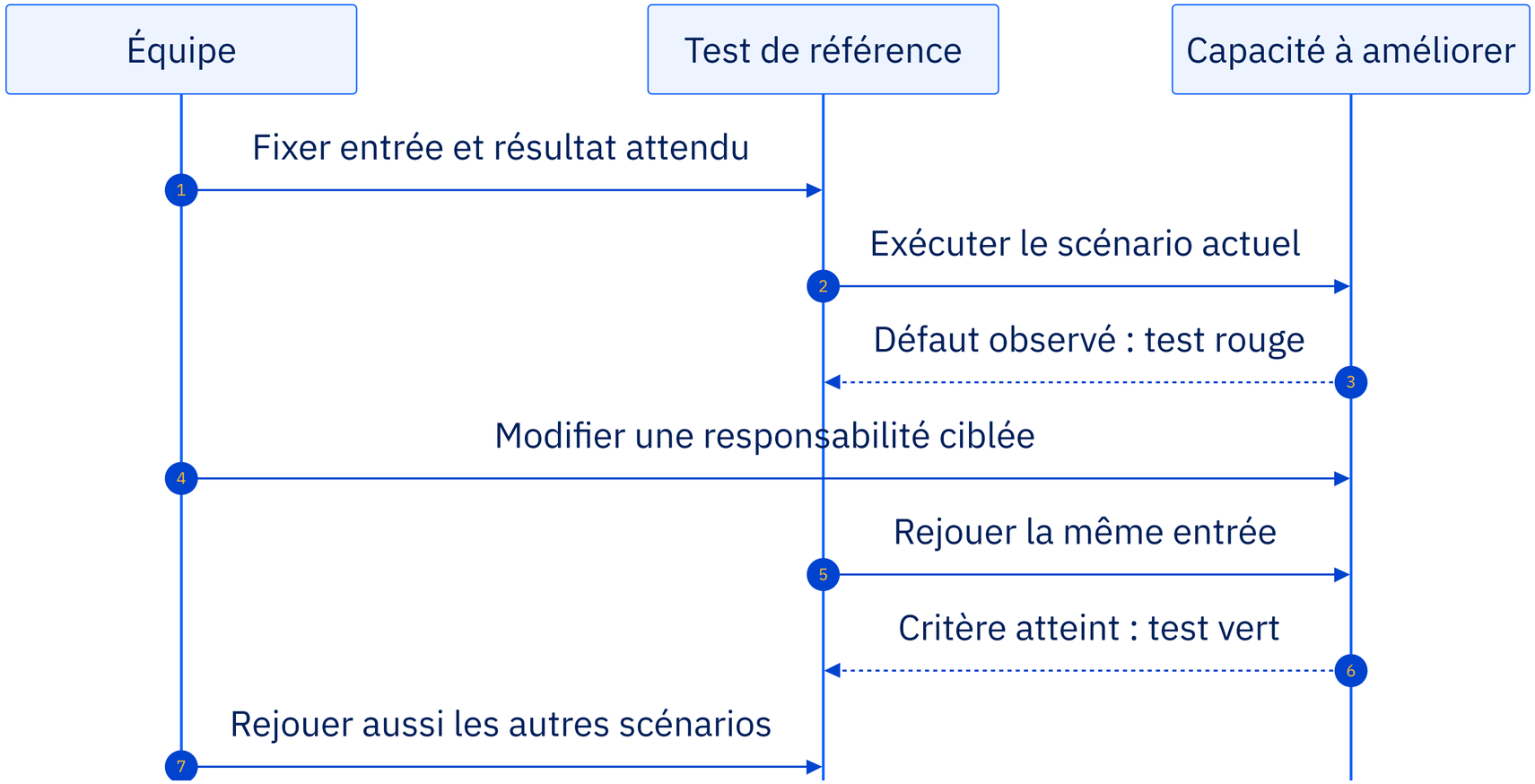
Exemple

« La question hybride ne trouve qu’un document. Après ajout d’une relation et d’un alias, elle doit produire un chemin sourcé à deux sauts. »

Une seule mesure suffit si elle est claire.

FLUX À IMPLÉMENTER · SÉQUENCE

Un même scénario prouve l’avant et l’après.



Écrire le critère avant la modification. Un test rouge reproduit le défaut ; la même entrée vérifie ensuite le progrès. Flèche pleine : appel ; pointillés : retour.

ATELIER · S6

Réduisez l’amélioration à une expérience mesurable.

01 Créer une fiche courte

Dans `docs/improvement.md` : problème, entrée, résultat actuel et critère attendu.

02 Écrire le test qui échoue

Préparer des fakes et les données minimales. Capturer sources, chemin ou nombre d’appels selon le défaut.

03 Modifier, comparer, documenter

Rejouer sans changer la question ni le critère. Garder le tableau avant/après et une limite réelle.

À CRÉER · docs/improvement.md · exemple

```
Question : lien entre deux objets connus
Avant : un passage, aucun chemin affiché
Critère : chemin sourcé à deux sauts
Modification : alias + relation vérifiée
Après : résultat du même scénario
Limite : alias absents hors du jeu connu

uv run pytest
uv run ruff check .
```

Vérifier. Le progrès est attribuable à une petite évolution. Si le test passe déjà avant, reformuler le défaut ou le critère.

Étapes détaillées et commandes copiables dans le README du TP.

IMPLÉMENTER SANS CASSER

Ajoutez la capacité au bon endroit, puis rejouez tout.

01

Règle ?

Dans le domaine, avec un test direct.

02

Orchestration ?

Dans un use case, via les ports existants.

03

Technologie ?

Dans un adaptateur sélectionné au bootstrap.

```
uv run pytest
uv run ruff check .
```

Documenter le résultat et une limite

Capsule C5

Changez Cohere ↔ Ollama uniquement avec `FILROUGE_LLM_PROVIDER`. Le domaine et les use cases ne bougent pas.

INSTALLATION FRAÎCHE

Votre projet doit repartir uniquement depuis Git.

1

Cloner ailleurs

Nouveau dossier, aucun environnement ou donnée locale cachée.

2

Installer

`uv sync --extra all --group dev`.

3

Reconstruire

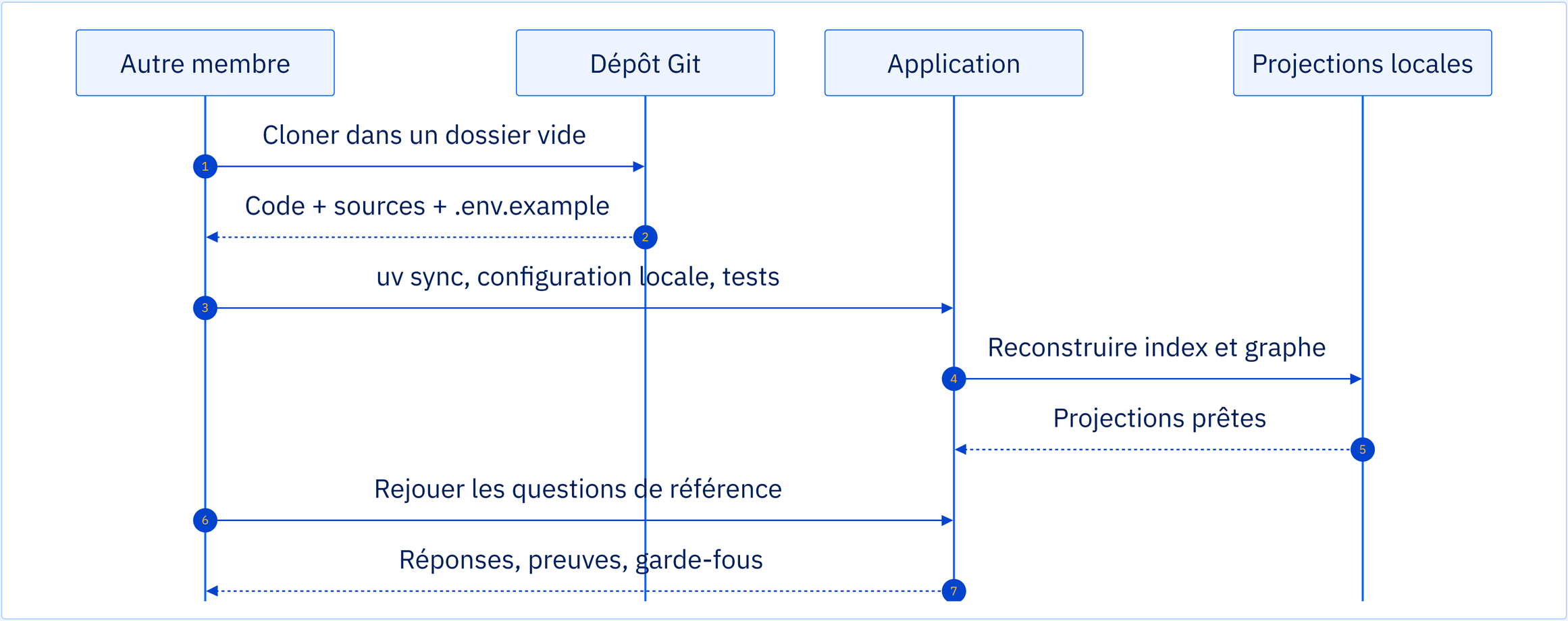
Corpus, index et graphe depuis les artefacts versionnés.

```
cp .env.example .env
# compléter localement sans commiter
uv run pytest
uv run ruff check .
uv run streamlit run app.py
```

Aucune copie de .venv ou data/

FLUX À IMPLÉMENTER · SÉQUENCE

La livraison commence dans un dossier vide.



Un autre membre suit uniquement le README et les fichiers suivis par Git, avec sa configuration locale. Flèche pleine : appel ; pointillés : retour.

ATELIER · S7

Transformez le README en procédure rejouable.

01 **Faire une installation croisée**

Un autre membre clone ailleurs. Aucun transfert de .venv, d’index, de graphe ou de fichier secret.

02 **Lister les commandes de reconstruction**

Documenter vos vrais scripts, leur ordre et les comptes attendus. Le profil fake doit démarrer sans clé.

03 **Répéter et corriger le premier blocage**

Rejouer les scénarios puis la démo. Corriger le README à l’endroit exact où l’autre membre s’est arrêté.

À EXÉCUTER · dans le clone neuf

```
uv sync --extra all --group dev
cp .env.example .env
# Compléter localement si nécessaire.
# Rejouer VOS commandes d’ingestion
# et de projection documentées.
uv run pytest
uv run ruff check .
uv run streamlit run app.py
```

Vérifier. Le projet repart sans fichier caché du premier poste. Tester un mode dégradé explicite si un service distant est indisponible.

Étapes détaillées et commandes copiables dans le README du TP.

CONTRÔLE DE LIVRAISON

Vérifiez ce que Git contient… et ce qu’il ne contient pas.

- ✓ `.env.example` décrit toutes les variables.
- ✓ Aucun secret ou token n’est suivi.
- ✓ Les données sont publiques ou synthétiques.
- ✓ Les scripts reconstruisent index et graphe.
- ✓ Le README donne un chemin de démarrage.
- ✓ Les limites sont écrites clairement.
- ✓ Les tests passent sur installation fraîche.
- ✓ Les quatre questions restent démontrables.

Capsule C7 · DRY

Extraire un composant UI réellement dupliqué dans une fonction testable. Ne refactorisez pas tout le projet à la veille de la livraison.

RÉPÉTITION GÉNÉRALE

Une démo courte, puis une décision défendue par personne.

3 MIN

Le besoin

Domaine, utilisateurs et question centrale.

5 MIN

Le parcours

Une question simple, une relation et un garde-fou.

3 MIN

Le recul

Architecture, mesure avant/après et limite principale.

- ✓ Chaque membre explique une décision.
- ✓ Un plan B fake existe si le live échoue.

PRÊT POUR S8

Votre prototype est devenu un projet défendable.

Vous avez prouvé

- une amélioration mesurable ;
- une installation reproductible ;
- un parcours complet et borné ;
- des limites connues.

Dernier contrôle

- Git propre et poussé ;
- tests et Ruff au vert ;
- README relu par un autre membre ;
- démonstration chronométrée.

```
git status
uv run pytest  &&  uv run ruff check .
git push
```

S6–S7 · CONSIGNES À GARDER PROJETÉES

À vous : améliorer, reproduire et défendre.

01 **S6 · Choisir et mesurer**

Un défaut, une entrée stable et un critère de réussite dans docs/improvement.md.

02 **S6 · Corriger avec une preuve**

Test rouge → évolution ciblée → test vert ; rejouer les quatre questions et noter la limite.

03 **S7 · Installer chez un autre membre**

Clone neuf, .env.example, commandes de reconstruction et tests ; corriger le README.

04 **S7 · Répéter la soutenance**

Démo chronométrée, sources visibles, un garde-fou ; une décision expliquée par personne.

À montrer avant de terminer

- Le même scénario avant et après.
- Une installation fraîche vérifiée par un autre membre.
- Une démo répétée et des limites écrites.

```
git status
uv run pytest
uv run ruff check .
```

Bloqué ? README de ce TP → « Pas à pas ». Garder une amélioration limitée ; le déploiement public reste facultatif.